FPT UNIVERSITY

PROJECT REPORT

DESIGN TDM 4-Channel Multiplexer/Demultiplexer — Verilog RTL Project

Team: 3

Class: IC2010

Team members: CAO HỮU AN

PHẠM MINH TÙNG

MAI THẾ TÂM

LÊ THẾ TRƯỜNG

PHAN ANH MINH

TRẦN MINH TRIẾT

Lecturer: Dr. Phạm Thế Vinh

HO CHI MINH CITY, NOVEMBER 2026

MỤC LỤC

1. SYSTEM OVERVIEW ..... - 2 -

1.1 Project Introduction ..... - 2 -

1.2 Main Operating Principles ..... - 2 -

2. BLOCK DIAGRAM AND DATA FLOW ..... - 2 -

2.1 System Block Diagram ..... - 2 -

3. REPOSITORY STRUCTURE AND MODULE INDEX ..... - 3 -

3.1 Project Directory Structure ..... - 3 -

3.2 RTL Module Index ..... - 3 -

4. TIMING CONTRACT & GLUE LOGIC ..... - 4 -

4.1 mux_select = (bit_count + 1)[4:3] Signal (Look-Ahead Logic) ..... - 4 -

4.2 load and byte_boundary = (bit_count[2:0] == 3'b111) Signals ..... - 5 -

4.3 demux_select = channel_select Signal (1-Clock-Cycle Delay) ..... - 5 -

4.4 Preload on Reset Logic ..... - 5 -

5. MODULE INTERFACE SPECIFICATIONS ..... - 5 -

5.1 tdm_counter Module ..... - 5 -

5.2 tdm_mux Module (4:1 Multiplexer) ..... - 6 -

5.3 tx_shift_reg Module (Parallel-to-Serial Shift Register) ..... - 6 -

5.4 rx_shift_reg Module (Serial-to-Parallel Shift Register / SIPO) ..... - 7 -

5.5 tdm_demux Module (Demultiplexer / Byte Router) ..... - 7 -

5.6 tdm_top Module (Top-Level Integration) ..... - 8 -

6. VERIFICATION AND TESTING ..... - 9 -

6.1 System Testing (python scripts/run_tests.py) ..... - 9 -

6.2 Comparison Against the Python Golden Model (Python Golden Model vs RTL) ..... - 9 -

7. VIVADO SYNTHESIS AND TIMING RESULTS ..... - 9 -

7.1 Hardware Resource Utilization ..... - 9 -

7.2 Design Timing Analysis (Design Timing Summary @ 100 MHz / 10 ns) ..... - 10 -

8. DESIGN COMPARISON AND TRADE-OFF ANALYSIS ..... - 10 -

8.1 Architecture Comparison: Counter-Driven (Current) vs. FSM-Driven (Traditional) ..... - 10 -

8.2 Interface Comparison: Single-Wire Serial TDM vs. Parallel Transmission (Parallel Data Bus) ..... - 11 -

8.3 Testing Method Comparison: Python Golden Model vs. Pure Verilog Testbench ..... - 11 -

9. LIMITATIONS AND FUTURE WORK ..... - 12 -

9.1 Current Limitations ..... - 12 -

9.2 Future Work ..... - 12 -

10. CONCLUSION ..... - 12 -

1. SYSTEM OVERVIEW

1.1 Project Introduction

This project implements the design of a 4-channel Time Division Multiplexing (TDM) Multiplexer and Demultiplexer using Verilog RTL. The system enables 8-bit data from 4 separate channels (CH0 to CH3) to be transmitted over a single serial data line (TDM_DATA) and accurately reconstructed at the corresponding outputs (CH0_OUT to CH3_OUT).

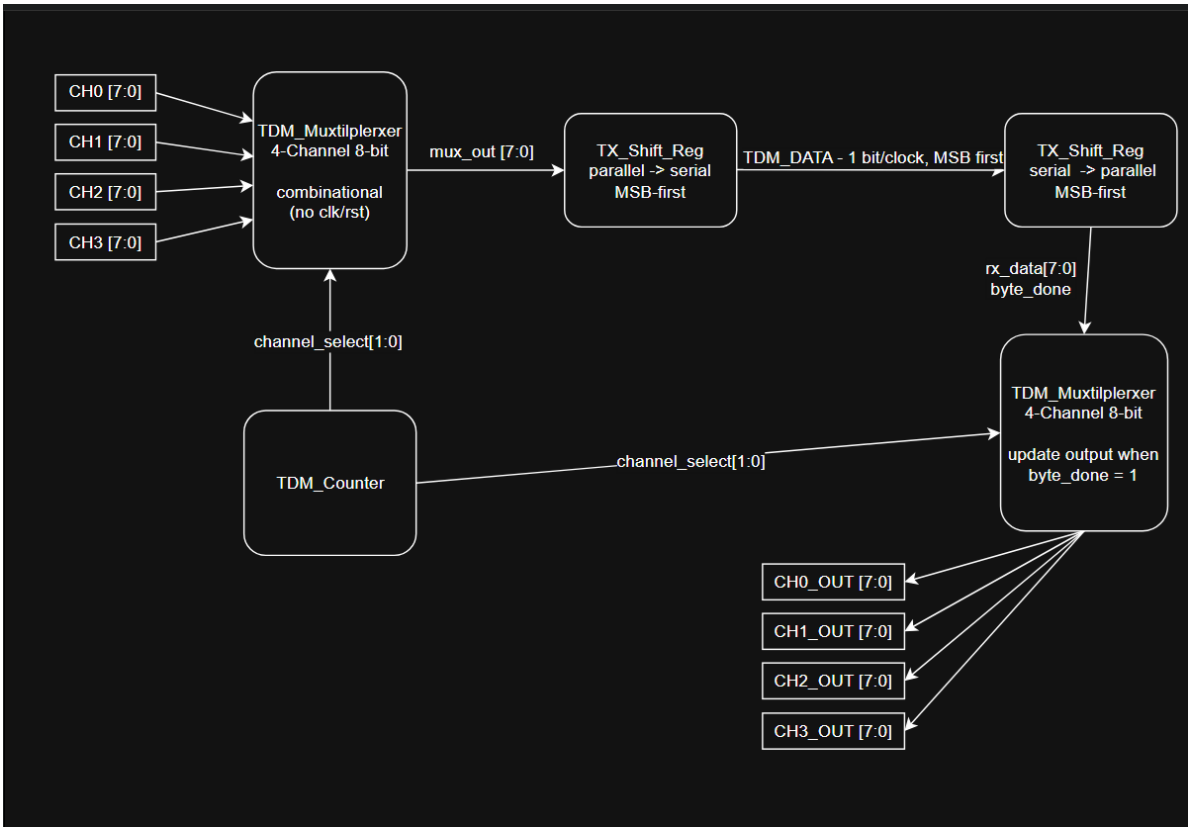
1.2 Main Operating Principles

- **Counter-driven / No FSM Architecture: The system is a fully cyclic datapath. It does not use a finite state machine (FSM), a complex handshake protocol, or any auxiliary synchronization mechanism.**
- **Time-slot Allocation:**
  - Each data byte (8 bits) occupies exactly 8 clock cycles for serial transmission, with the most significant bit transmitted first (MSB-first).
  - A transmission frame consists of 4 consecutive bytes in a fixed order: CH0 -> CH1 -> CH2 -> CH3 (a total of 32 clock cycles per frame).
- **Single Time Synchronization Source: The entire transmit/receive system operates based on a single free-running 5-bit counter (bit_count, counting from 0 -> 31 -> 0). The receiver automatically identifies which channel a received byte belongs to and writes it to the corresponding output register without requiring an additional synchronization signal.**
- **System Latency:**
  - **Cold Start: After reset, the first valid frame is available after 33 clock cycles.**
  - **Mid-stream Change: Takes 32 to 40–41 clock cycles (the worst case is 41 cycles: 31 cycles waiting for the channel load slot + 8 serial transmission cycles + 2 output-latching cycles).**

2. BLOCK DIAGRAM AND DATA FLOW

2.1 System Block Diagram

Data is processed through the main functional blocks: parallel input data passes through the MUX, is converted to serial form, transmitted over the TDM_DATA line, converted back to parallel form at the receiver, and routed through the DEMUX to the output registers.



3. REPOSITORY STRUCTURE AND MODULE INDEX

3.1 Project Directory Structure

- **src/ (Synthesizable RTL):**
  - tdm_counter.v: Free-running 5-bit counter (the main timing source).
  - tdm_mux.v: 4-to-1 multiplexer.
  - tx_shift_reg.v: Parallel-to-serial shift register.
  - rx_shift_reg.v: Serial-to-parallel shift register.
  - tdm_demux.v: Byte router to the CHX_OUT output registers.
  - tdm_top.v: Top-level integration module + timing glue layer.
- **techbench/: Contains the system testbenches (tb_tdm_top.v, tb_tdm_top_filebased.v) and explanatory documentation.**
- **golden_model/: Python reference model and automatic comparison scripts (run_verification.py, compare_output.py...).**
- **input/: Contains input data vectors (input_vectors.hex, random_vectors.hex).**
- **scripts/: Contains the testbench execution script (run_tests.py) and the Vivado synthesis TCL file (synth.tcl).**
- **constraint/: Clock timing constraint file (constraints.xdc).**

3.2 RTL Module Index

No.	MODULE	SOURCE FILE	ROLE

1	Tdm_counter	Tdm_counter.v	The sole timing source. Free-running 5-bit bit_count counter (0 -> 31 -> 0), directly outputs the channel-select signal channel_select = bit_count[4:3].
2	Tdm_mux	Tdm_mux.v	4-to-1 combinational multiplexer. Selects the current channel's 8-bit data byte for serial conversion.
3	Tx_shift_reg	Tx_shift_reg.v	Parallel-to-serial shift register (MSB-first). Loads a new byte when the load pulse occurs; otherwise, shifts left by 1 bit/clock.
4	Rx_shift_reg	Rx_shift_reg.v	Serial-to-parallel shift register (MSB-first). Shifts continuously; when byte_boundary is reached, latches the data into rx_data and generates the byte_done pulse.
5	Tdm_demux	Tdm_demux.v	Demultiplexer that routes the complete byte to the correct CHX_OUT register, controlled by the byte_done pulse.
6	Tdm_top	Tdm_top.v	Top-level module. Connects 5 sub-modules and contains the timing glue logic that generates precisely phase-shifted control signals.

4. TIMING CONTRACT & GLUE LOGIC

Because the tx_shift_reg and rx_shift_reg shift registers operate synchronously with the clock, loaded data only takes effect at the next clock edge. Three small glue-logic components in tdm_top are responsible for shifting the raw control signals from the counter into precisely phase-aligned signals without modifying the 5 sub-modules:

4.1 mux_select = (bit_count + 1)[4:3] Signal (Look-Ahead Logic)

- **Purpose: Look one time step ahead so that tx_shift_reg can load the next channel's byte exactly one clock cycle before the bit-shifting process begins.**
- **Importance: Without this look-ahead mechanism, the output channel sequence would be permanently rotated by one position (input data AA/BB/CC/DD would appear at the outputs as DD/AA/BB/CC).**

4.2 load and byte_boundary = (bit_count[2:0] == 3'b111) Signals

- **Purpose: Trigger the pulse on the last bit of the current byte (3'b111), rather than on the first bit of the next byte, ensuring that loading (TX) and data latching (RX) occur with complete timing accuracy.**

4.3 demux_select = channel_select Signal (1-Clock-Cycle Delay)

- **Purpose: Delay the channel-select signal by one clock cycle to keep the decoded channel selector synchronized with the byte_done pulse (which is itself a pulse latched one cycle later by rx_shift_reg).**

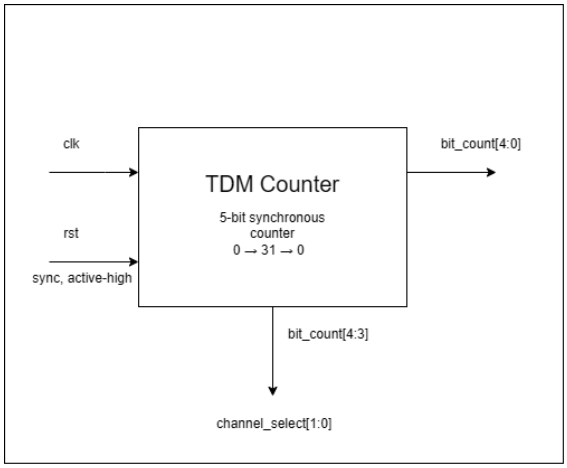
4.4 Preload on Reset Logic

- In tx_shift_reg, when the reset signal is asserted, the register preloads the mux_out value instead of clearing to 8'h00. Since bit_count is held at 0 during reset, mux_select already points to CH0, allowing CH0 to become the first channel to complete a transmission cycle immediately after reset is released.

5. MODULE INTERFACE SPECIFICATIONS

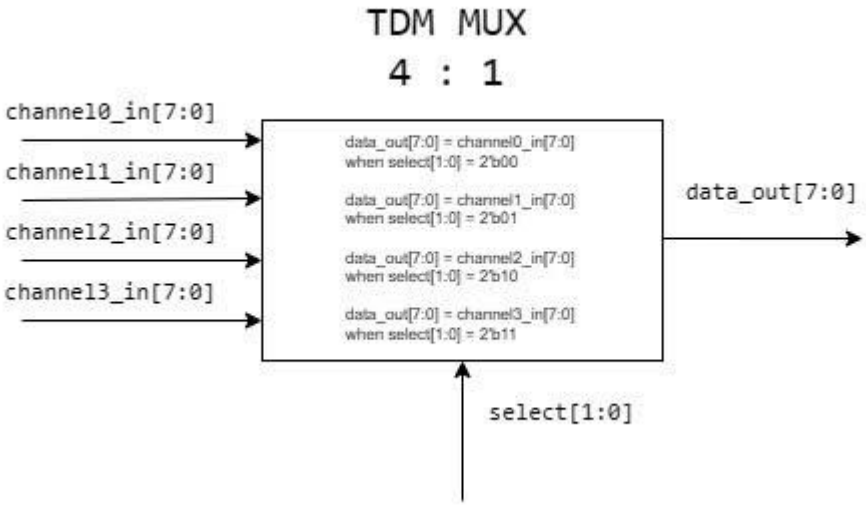
5.1 tdm_counter Module

- **Inputs:**
  - clk (1-bit): Common system clock.
  - rst (1-bit): Synchronous, active-high reset signal.
- **Outputs:**
  - bit_count (5-bit): Free-running counter from 0 -> 31 -> 0, the main timing source of the system.
  - channel_select (2-bit): Value of bit_count[4:3] (raw/unregistered).



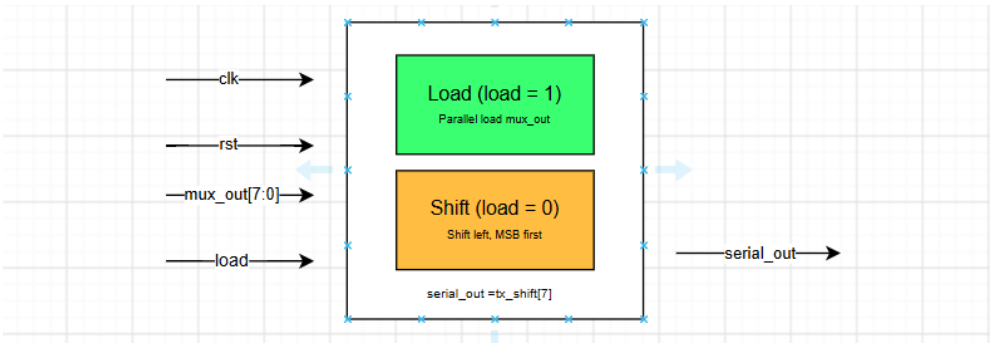
5.2 tdm_mux Module (4:1 Multiplexer)

- **Inputs:**
  - CH0 .. CH3 (8-bit x 4): Four parallel input channels.
  - channel_select (2-bit): Channel-select signal (00 -> CH0, 01 -> CH1, 10 -> CH2, 11 -> CH3).
- **Outputs:**
  - mux_out (8-bit): Data byte from the selected channel (implemented entirely using combinational logic).



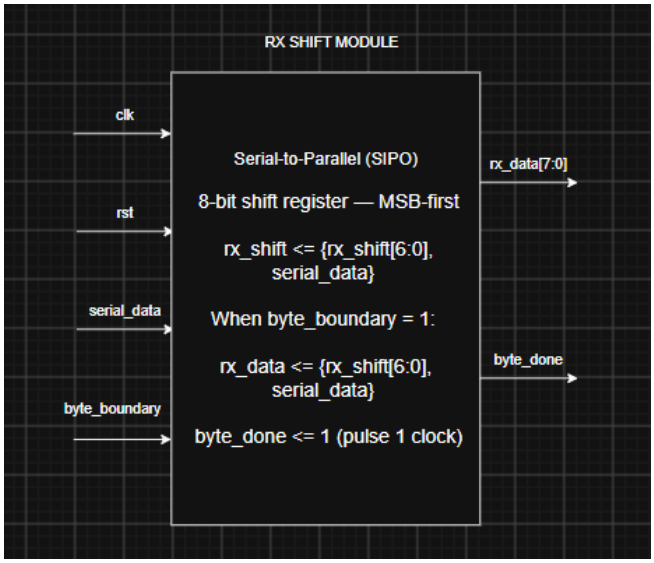
5.3 tx_shift_reg Module (Parallel-to-Serial Shift Register)

- **Inputs:**
  - clk, rst (1-bit): Synchronous active-high clock and reset.
  - mux_out (8-bit): Byte data loaded from tdm_mux.
  - load (1-bit): Data-load trigger pulse (1 pulse/byte).
- **Outputs:**
  - serial_out (1-bit): Serial output (tx_shift[7], the current MSB).
- **Logic priority: RESET > LOAD > SHIFT.**



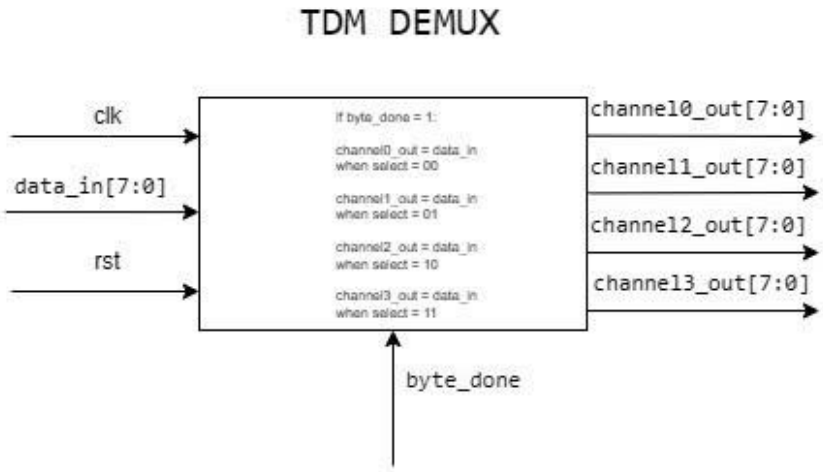
5.4 rx_shift_reg Module (Serial-to-Parallel Shift Register / SIPO)

- **Inputs:**
  - clk, rst (1-bit): Synchronous clock and reset.
  - serial_data (1-bit): Serial bit sampled once per clock cycle from TDM_DATA.
  - byte_boundary (1-bit): Pulse indicating the beginning/end of a byte (역할 equivalent to load on the TX side).
- **Outputs:**
  - rx_data (8-bit): Fully reconstructed byte (MSB-first).
  - byte_done (1-bit): Byte-completion pulse (appears one cycle after byte_boundary).



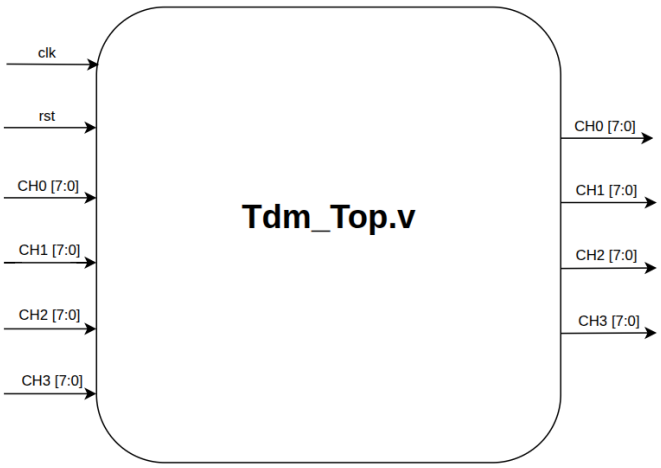
5.5 tdm_demux Module (Demultiplexer / Byte Router)

- **Inputs:**
  - clk, rst (1-bit): Synchronous active-high clock and reset.
  - rx_data (Input, 8-bit): Data byte received from rx_shift_reg.
  - channel_select (2-bit): Select signal for the output channel to be updated.
  - byte_done (1-bit): Write-enable signal (update gate; write when high).
- **Outputs:**
  - CH0_OUT .. CH3_OUT (8-bit x 4): Reconstructed output channels (retain their previous values if not selected).



5.6 tdm_top Module (Top-Level Integration)

- **Inputs:**
  - clk, rst (1-bit): System clock and reset.
  - CH0 .. CH3 (8-bit x 4): Four parallel input data channels.
- **Outputs:**
  - TDM_DATA (1-bit): The single serial data transmission line.
  - CH0_OUT .. CH3_OUT (8-bit x 4): Four reconstructed output data channels.



6. VERIFICATION AND TESTING

6.1 System Testing (python scripts/run_tests.py)

- **Result: Passed 38 / 38 test cases on both Icarus Verilog (iverilog) and Vivado XSIM.**
- **Coverage (3 independent testing layers):**
    1. Layer 1: Frame-by-frame round-trip transmission and reception testing, including boundary cases (0x00, 0xFF) and a mid-stream reset.
    2. Layer 2: Bit-level comparison of all 32 bits on the TDM_DATA line against the expected MSB-first bit sequence (ensuring that TX and RX errors cannot cancel each other out and remain hidden).
    3. Layer 3: Regression testing with 20 consecutive random frames (back-to-back).
- **Simulator Portability: tb_tdm_top releases reset with a #1 delay after the clock edge to avoid assignment-order conflicts within the same time step as the DUT.**

6.2 Comparison Against the Python Golden Model (Python Golden Model vs RTL)

The tb_tdm_top_filebased.v file reads input vectors from input/*.hex, activates tdm_top, and writes the results to a file. The run_verification.py file compares each vector against an independent Python model:

7. VIVADO SYNTHESIS AND TIMING RESULTS

- **Synthesis Tool: Xilinx Vivado 2020.2**
- **Target FPGA Family: Kintex-7 (xc7k70tfbv676-1)**
- **Target Clock Frequency: 100 MHz (10 ns period constraint in constraints.xdc)**

7.1 Hardware Resource Utilization

Hardware Resource	Used	Available	Utilization (%)
Slice LUTs	**22**	41,000	0.05%
Slice Registers (Flip-Flops)	**63**	82,000	0.08%
Bonded IOB	**67**	300	22.33%
Block RAM (BRAM)	**0**	135	0%
DSP Blocks	**0**	240	0%
BUFGCTRL (Clock buffer)	**1**	32	3.13%

Primitive Breakdown:

- 63 FDREs (fully synchronous design, with no latches).
- LUT logic: 15 LUT6, 6 LUT3, 3 LUT5, 2 LUT4, 1 LUT2, 1 LUT1.
- I/O primitives: 34 IBUF, 33 OBUF.
- Clock buffer: 1 BUFG.
- Observation: I/O pins account for the largest proportion (22.33%) because the design transmits 48-bit parallel input and 48-bit parallel output data, along with clock/reset/serial signals; the design uses no BRAM or DSP resources.

7.2 Design Timing Analysis (Design Timing Summary @ 100 MHz / 10 ns)

Timing Check	Worst Slack	Number of Failing Endpoints	Total Endpoints	Status
Setup (WNS)	**+6.574 ns**	0	104	**Met (TIMING MET)**
Hold (WHS)	**+0.159 ns**	0	104	**Met (TIMING MET)**
Pulse Width (WPWS)	**+4.600 ns**	0	65	**Met (TIMING MET)**

Timing Performance Evaluation:

- All user-specified timing constraints are met (TNS = 0.000 ns, THS = 0.000 ns, TPWS = 0.000 ns).
- Based on the setup slack margin (+6.574 ns), the estimated minimum clock period is 10 - 6.574 = 3.43 ns, equivalent to an estimated maximum operating frequency of approximately 290 MHz.

8. DESIGN COMPARISON AND TRADE-OFF ANALYSIS

In digital communication system design, the choice of hardware architecture and testing methodology plays a decisive role in performance, chip resources, and reliability. The following are the project's key comparison and trade-off analyses:

8.1 Architecture Comparison: Counter-Driven (Current) vs. FSM-Driven (Traditional)

- **Counter-driven architecture (used in this project):**

    - *Advantages: The datapath is fully cyclic, completely eliminating invalid states or the risk of stuck states commonly found in FSMs. It maximizes hardware resource savings (using only 22 LUTs and 63 FFs).*
    - *Disadvantages: Less flexible when changing complex protocols mid-stream without a reset.*
- **Finite State Machine (FSM)-driven architecture:**
    - *Advantages: Easier to manage protocols with complex handshaking mechanisms or multiple operating modes.*
    - *Disadvantages: Requires additional resources for state-transition logic and can introduce logic bugs in boundary states if not designed carefully.*

8.2 Interface Comparison: Single-Wire Serial TDM vs. Parallel Transmission (Parallel Data Bus)

- **TDM interface (Time Division Multiplexing - used in this project):**
    - *Advantages: Extremely efficient use of FPGA I/O pins (only one TDM_DATA line is used to transmit all 4 channels instead of requiring 32 separate lines). Reduces PCB routing area.*
    - *Disadvantages: Introduces serialization latency (8 clock cycles are required to transmit one byte) and each channel's instantaneous bandwidth is shared over time.*
- **Traditional parallel transmission:**
    - *Advantages: High instantaneous data-transfer speed, with no serial-to-parallel conversion cycles required.*
    - *Disadvantages: The number of FPGA IOB pins increases dramatically, causing hardware overload when the number of channels becomes large.*

8.3 Testing Method Comparison: Python Golden Model vs. Pure Verilog Testbench

- **Python Golden Model (used in this project):**
    - *Advantages: Fully automates the generation of a large number of random vectors (200+ vectors) and performs exact bit-for-bit comparisons on every clock cycle, without requiring complex manual $display or if/else statements in Verilog.*
    - *Disadvantages: Requires a multi-language execution environment (Python combined with Icarus Verilog or Vivado XSIM).*
- **Traditional pure Verilog testbench:**
    - *Advantages: Runs entirely independently within the HDL simulator, without depending on external tools.*
    - *Disadvantages: Difficult to write complex random testing scenarios, and manual result checking can easily miss edge cases.*

9. LIMITATIONS AND FUTURE WORK

9.1 Current Limitations

1. **Channel order when data changes mid-stream: The channel order after a mid-stream data change is not yet completely fixed; this can be addressed by adding four input buffer registers that update only when bit_count wraps around to 0.**
2. **Rare boundary cases: Some boundary cases have not been fully covered, such as back-to-back resets, extremely short reset pulses, or clock jitter.**
3. **Hard-coded Parameters: The number of channels and data width are currently fixed; they can be generalized using the NUM_CHANNELS / DATA_WIDTH parameters together with a generate loop.**
4. **Error-Checking Features: There is currently no error-detection mechanism or frame synchronization marker. Adding one parity bit per byte and a small marker-code detector is entirely feasible.**
5. **Out of Scope: CRC error detection, clock recovery, and clock-domain crossing for physically separate transmit/receive clocks are not yet supported.**

9.2 Future Work

1. **Add a Vivado Timing Report File: Add a timing report file (.rpt) to the results/ directory and apply I/O delay constraints so that input and output paths are also fully analyzed for timing.**
2. **Configuration on a Target Board: Select a specific FPGA board, set the actual clock frequency, and configure pin assignments in the constraint/constraints.xdc file.**
3. **Implement Input Buffer Registers: Add input buffer registers as described and extend the golden model to verify them.**
4. **Complete Boundary-Case Coverage: Add testbenches to cover the remaining boundary cases (short reset pulses and back-to-back resets).**

10. CONCLUSION

The design project for the Time Division Multiplexing 4-Channel Multiplexer/Demultiplexer has successfully achieved its stated objectives in terms of technical implementation, functionality, and hardware performance:

- **Architecture and Design: The system has demonstrated the effectiveness of a synchronous counter-driven architecture combined with precise glue logic, completely eliminating the complexity of a finite state machine (FSM) while maintaining a smooth cyclic data flow. The design significantly optimizes the**

number of I/O pins by transmitting four 8-bit data channels over a single line (TDM_DATA).

- **Testing and Verification: The comprehensive testing environment, with 38/38 system test cases combined with the Python Golden Model running 200/200 random vectors with exact bit-for-bit matches, has confirmed the correctness, stability, and high reliability of the Verilog RTL source code.**
- **Hardware Synthesis: Synthesis results on the Kintex-7 FPGA family using Vivado 2020.2 show that the design is highly resource-efficient (using only 22 LUTs and 63 FFs, with no BRAM or DSP required) and comfortably meets the timing constraints at 100 MHz (with a WNS margin of +6.574 ns and the potential for stable operation at frequencies up to approximately 290 MHz).**

Team Members' Contribution

Class: IC2010		Team: 3		Contribution
#	Student ID	Student Name	Task	%
1	SE205188	CAO HỮU AN	TDM_DEMUX Module + REPORT	16%
2	SE204386	PHẠM MINH TÙNG	TDM_MUX Module + TDM_TOP + TESTBENCH + Integration	20%
3	SE205068	MAI THẾ TÂM	TDM_Counter Module	16%
4	SE202014	LÊ THẾ TRƯỜNG	TX_SHIFT_REG Module	16%
5	SE204511	TRẦN MINH TRIẾT	GOLDEN MODEL	16%
6	SE204664	PHAN ANH MINH	RX_SHIFT_REG Module	16%
Total:				**100%**

References

[1] IEEE Standard for Verilog Hardware Description Language

[2] Xilinx Kintex-7 FPGA Data Sheet: Overview

[3] Vivado Design Suite User Guide: Logic Simulation

[4] Vivado Design Suite User Guide: Synthesis

[5] Python HDL Verification Frameworks and Golden Model Methodologies

[6] Advanced Digital Design with the Verilog HDL

[7] RTL Modeling with the IEEE Standard for Verilog

[8] Digital Integrated Circuit Design: From VLSI Architectures to CMOS Fabrication

[9] Time-Division Multiplexing (TDM) Architecture and Applications in Digital Communications

[10] Icarus Verilog (iverilog) Open-Source Simulation Tool Documentation